



第8章

错误和异常



错误

- 代码运行前的语法或逻辑错误。
 - **语法错误**是指源代码中的拼写不符合解释器和编译器所要求的语法规则，一般集成开发工具中都会直接提示语法错误，编译时提示SyntaxError。必须在程序执行前改正，否则程序无法运行。
 - **逻辑错误**是程序代码可执行，但执行结果不符合要求。例如求两个数中最大数，返回的结果却是最小数。



语法错误(**SyntaxError**)

File "/Users/bxliu/Code/pythonProject/reg.py", line 15

```
match = re.findall(reg。 data)
```

^

SyntaxError: invalid character in identifier

进程已结束,退出代码1

- 语法错误 **SyntaxError : invalid syntax/character**
- 无法正常编译，针对语法错误，python解析器会输出错误的那一行，并且在最先找到的错误的位置标记了一个箭头。



常见语法错误

1. 需要使用英文符号的地方用了中文符号，例如逗号、冒号、括号等
2. 变量、函数等命名不符合标识符规范
3. 条件语句、循环语句、函数定义后面忘了写冒号
4. 位于同一层级的语句缩进不一致/缩进字符错误
5. 判断两个对象相等时，使用一个等号而不是两个等号
6. 语句较为复杂时，括号的嵌套关系错误，少了或多了左/右括号
7. 函数定义时，不同类型参数之间的顺序不符合要求
8. 赋值错误
9. 循环逻辑错误



对应的错误提示

1. `SyntaxError: invalid character in identifier`
2. `SyntaxError: invalid syntax`
3. `SyntaxError: invalid syntax`
4. `IndentationError: expected an indented block` / `IndentationError: unexpected indent`
5. `SyntaxError: invalid syntax`
6. `SyntaxError: unmatched ')'`
7. `SyntaxError: non-default argument follows default argument`
8. `SyntaxError: can't assign to literal`
9. `SyntaxError: 'break' outside loop` / `SyntaxError: 'continue' not properly in loop`



练习时间

- 15mins
- 根据提供的示例代码TestError.py
 - 删除对应行的#注释符号，注释原行代码，观察错误提示
- 思考问题：你在编程时候遇到哪种情况最多



异常(Exception)

- 程序语法可以通过检查
- 但当Python检测到一个运行异常时，解释器就会指出当前代码已无法继续执行下去
- 用**异常对象(exception object)**来表示异常情况。
- 如果异常对象并未被处理或捕捉，程序就会用**回溯 (traceback , 一种错误信息)**终止执行



Traceback & Exception

Traceback (most recent call last):

File `"/Users/bxliu/Code/pythonProject/reg.py"`, line 15, in
<module>

```
match = re.findall(reg.data)
```

AttributeError: 'str' object has no attribute 'data'



错误和异常



错误：

- 是指代码不符合解释器或者编译器语法

异常

- 是指不完整、不合法输入，或者计算出现错误





异常 (exception)

- 异常是指程序运行时引发的错误。
- 异常有不同的类型，发生异常后，其类型名称会被打印出来。
- 引发异常的原因有很多，例如名字错误、类型错误、除零、下标越界、文件不存在、网络异常、字典键错误、磁盘空间不足，等等。
- 用户也可以自定义异常



异常对象(exception object)

- 下面列出了常用的异常类

异常	描述
NameError	尝试访问一个没有声明的变量
ZeroDivisionError	除数为 0
SyntaxError	语法错误
IndexError	索引超出序列范围
KeyError	请求一个不存在的字典关键字
IOError	输入输出错误（例：读取不存在的文件）
AttributeError	尝试访问未知的对象属性
ValueError	传给函数的参数类型不正确（例：给 <code>int()</code> 函数传入字符串）
AssertionError	断言异常



扩展知识

Python继承（不做要求）



面向对象的编程 Object Oriented

- 对象的含义是指具体的某一个事物，即在现实生活中能够看得见摸得着的事物。
- 一切事物皆为对象
 - 对象可按其性质划分为类，对象成为类的实例



面向对象的编程 Object Oriented

- 在面向对象程序设计中，对象所指的是计算机系统中的某一个成分。
- 对象则是数据和动作的结合体，用属性和行为来进行描述。对象不仅能够进行操作，同时还能够及时记录下操作结果。



面向对象的编程 Object Oriented

- 面向对象的编程带来的主要好处之一是代码的重用
 - 实现各种重用的方法之一是通过继承机制。
 - 继承完全可以理解成类之间的父类和子类型关系。
 - 如猫类，猫是动物；猫类继承于动物类，动物类为父类也是所有动物的基类；猫类是动物类的子类，也是动物类的派生类。

封装



- 封装

- 把客观事物封装成抽象的类，并且类可以把自己的数据和方法只让可信的类或者对象操作，对不可信的进行信息隐藏。
- 一个类就是一个封装了数据以及操作这些数据的数据的逻辑实体。在一个对象内部，某些代码或某些数据可以是私有的，不能被外界访问。
- 通过这种方式，对象对内部数据提供了不同级别的保护，以防止程序中无关的部分意外的改变或错误的使用了对象的私有部分。



继承

- 继承
 - 可以让某个类型的对象获得另一个类型的对象的属性的方法。
 - 支持按级分类的概念。
 - 它可以使用现有类的所有功能，并在无需重新编写原来的类的情况下对这些功能进行扩展。



继承

- 通过继承创建的新类称为“子类”或“派生类”，被继承的类称为“基类”、“父类”或“超类”。
- 继承的过程，就是从一般到特殊的过程。要实现继承，可以通过“继承”（Inheritance）和“组合”（Composition）来实现
- 继承概念的实现方式有二类：实现继承与接口继承。实现继承是指直接使用父类的属性和方法而无需额外编码的能力；接口继承是指仅使用属性和方法的名称、但是子类必须提供实现的能力。



- 多态

- 是指一个类实例的相同方法在不同情形有不同表现形式。
- 多态机制使具有不同内部结构的对象可以共享相同的外部接口。
- 虽然针对不同对象的具体操作不同，但通过一个公共的类，它们（那些操作）可以通过相同的方式予以调用。

继承

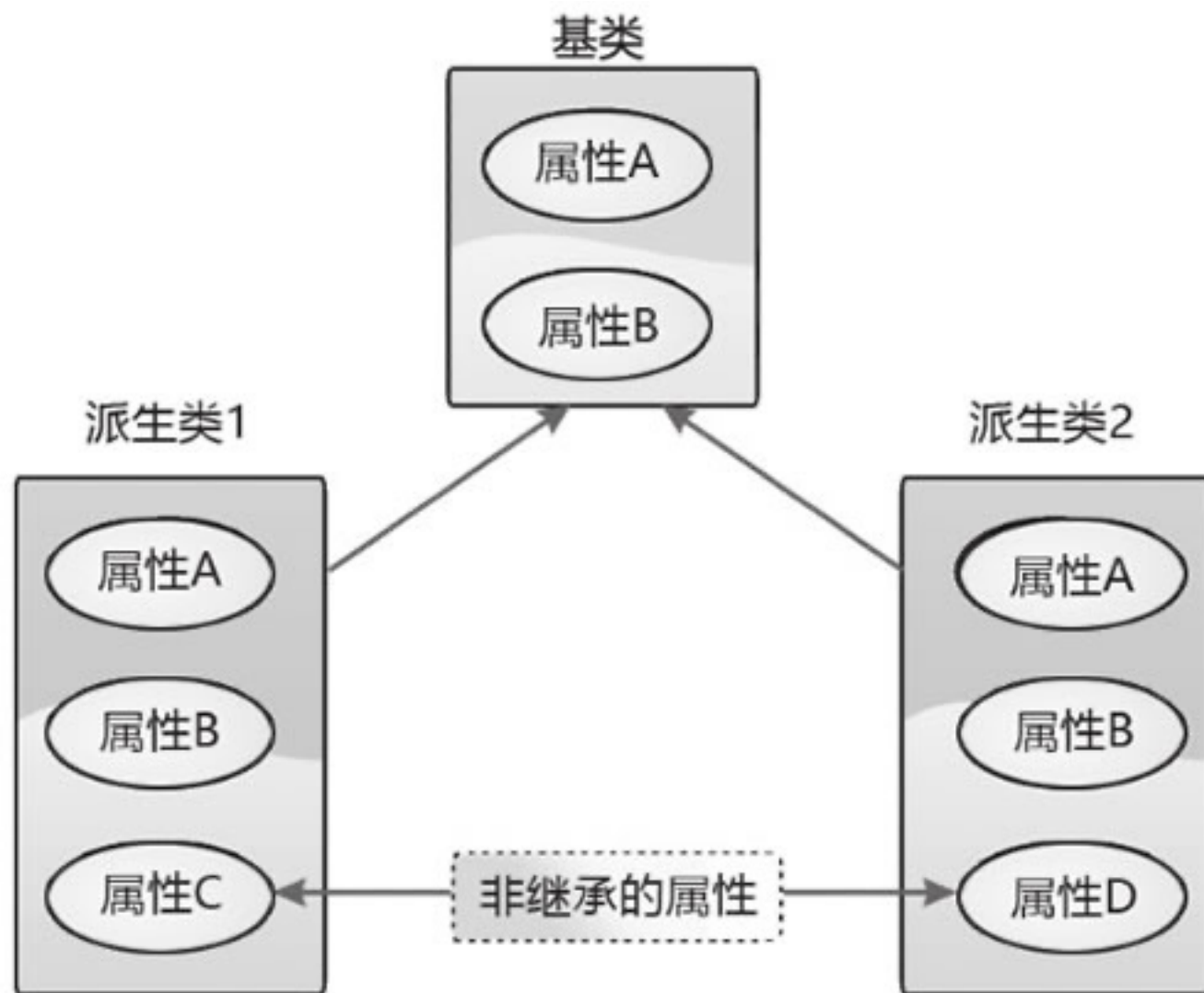


- **什么时候使用：**

- 假如需要定义几个类，而类与类之间有一些公共的属性和方法，可以把相同的属性和方法作为基类的成员，
- 而特殊的方法及属性则在子类中定义。这样子类只需要继承基类（父类），子类就可以访问到基类（父类）的属性和方法了。
- 如：猫，狗 都属于动物，它们行为相似性高，有个名字，都会叫，会吃，有四条腿。



- 类：可以理解是一个模板，通过它可以创建出无数个具体实例。比如，前面编写的 Cat 表示的猫这个物种，通过它可以创建出无数个实例来代表各种不同特征的猫（这一过程又称为类的实例化）。
- 对象：类并不能直接使用，通过类创建出的实例（又称对象）才能使用。这有点像汽车图纸和汽车的关系，图纸本身（类）并不能为人们使用，通过图纸创建出的一辆辆车（对象）才能使用。
- 属性：类中的所有变量称为属性。例如，Animal 这个类中，name 都是这个类拥有的属性。
- 方法：类中的所有函数通常称为方法。不过，和函数所有不同的是，类方法至少要包含一个 self 参数（后续会做详细介绍）。例如，Animal 类中，setname()、eat()都是这个类所拥有的方法，类方法无法单独使用，只能和类的对象一起使用。





类的函数

- 类的方法与普通的函数只有一个特别的区别——它们必须有一个额外的第一个参数名称`self`，代表对实例的引用
- 请注意区分**类（Class）**和**实例（Instance）**

```
class Test: #类定义
    def prt(self):
        print(self)
        print(self.__class__)
t = Test() #初始化实例
t.prt()
```



示例

```
class Employee:
```

'所有员工的基类'

empCount = 0 #类变量，它的值将在这个类的所有实例之间共享。你可以在内部类或外部类使用Employee.empCount 访问。

```
def __init__(self, name, salary): #构造函数
```

```
    self.name = name
```

```
    self.salary = salary
```

```
    Employee.empCount += 1
```

```
def displayCount(self):
```

```
    print("Total Employee %d" %  
Employee.empCount)
```

```
def displayEmployee(self):
```

```
    print("Name : ", self.name, ", Salary: ",  
self.salary)
```

```
A = Employee("1",200) #创建实例对象
```

```
A.displayEmployee()
```



访问属性

- 使用点号 . 来访问对象的属性
- `getattr(obj, name[, default])` : 访问对象的属性。
- `hasattr(obj,name)` : 检查是否存在一个属性。
- `setattr(obj,name,value)` : 设置一个属性。如果属性不存在 , 会创建一个新属性。
- `delattr(obj, name)` : 删除属性。



Python内置类属性

- `__dict__`: 类的属性 (包含一个字典, 由类的数据属性组成)
- `__doc__`: 类的文档字符串
- `__name__`: 类名
- `__module__`: 类定义所在的模块 (类的全名是 '`__main__.className`', 如果类位于一个导入模块 `mymod` 中, 那么 `className.__module__` 等于 `mymod`)
- `__bases__`: 类的所有父类构成元素 (包含了一个由所有父类组成的元组)



类的魔法函数定义

- `__init__`
- `__str__`
- `__del__`
- 方法名如果是`__xxxx__()`的，那么就有特殊的功能，因此叫做“魔法”函数



构造函数

- 用于创建对象时使用，每当创建一个类的实例对象时，自动调用它

`def __init__(self,...):`

代码块



名称函数

- 返回一个对象的描述信息
- `def __str__(self):`

代码块



析构函数

- `__del__`在对象销毁的时候被调用，当对象不再被使用时，`__del__`方法运行：
- `def __del__(self):`
 代码块

继承



- class 派生类名(基类名)

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname
    def printname(self):
        print(self.firstname, self.lastname)
class Student(Person):
    pass
```

Demo



```
class Animal: # 基类
    def __init__(self):
        self.name = "No name"

    def setname(self, name="No name"):
        self.name = name

    def eat(self):
        print(self.name + ' is Eating')

    def run(self):
        print(self.name + ' is Running')
```

```
class Cat(Animal): # 派生类
    pass
```

```
class Dog(Animal):
    pass
```

```
class Fish(Animal):
    def swim(self):
        print(self.name + ' is Swimming')
```

产生子类对象

```
fish = Fish()
fish.swim()
fish.eat()
print(fish.name)
```

```
kitty = Cat()
kitty.setname("K")
kitty.eat()
print(kitty.name)
```

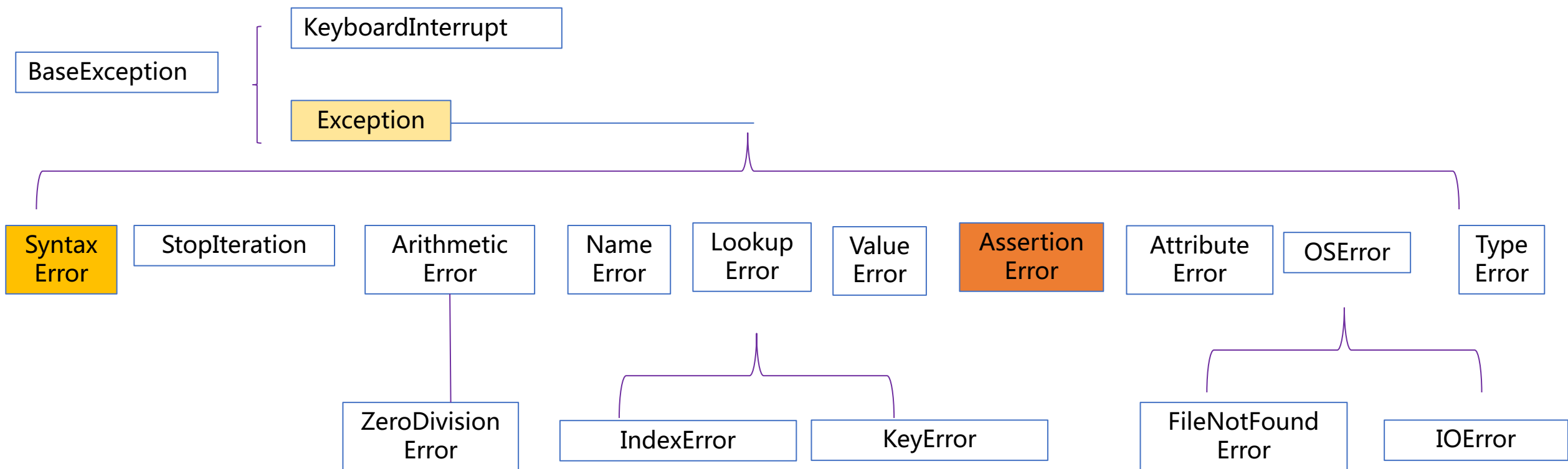


异常对象(exception object)

- 在 Python 中，所有异常必须为一个派生自 [BaseException](#) 的类的实例。
 - BaseException是所有异常的基类
 - Exception是python中所有普通异常的基类
 - 所有内置的非系统退出类异常都派生自此类
 - 所有用户自定义异常也应当派生自此类
 - Warning也是某种形式的异常



异常对象(exception object)



```

+-- Exception
+-- StopIteration
+-- StopAsyncIteration
+-- ArithmeticError
|   +-- FloatingPointError
|   +-- OverflowError
|   +-- ZeroDivisionError
+-- AssertionError
+-- AttributeError
+-- BufferError
+-- EOFError
+-- ImportError
|   +-- ModuleNotFoundError
+-- LookupError
|   +-- IndexError
|   +-- KeyError
+-- MemoryError
+-- NameError
|   +-- UnboundLocalError
+-- OSError
|   +-- BlockingIOError
|   +-- ChildProcessError
|   +-- ConnectionError
|       +-- BrokenPipeError
|       +-- ConnectionAbortedError
|       +-- ConnectionRefusedError
|       +-- ConnectionResetError
|   +-- FileExistsError
|   +-- FileNotFoundError
|   +-- InterruptedError
|   +-- IsADirectoryError
|   +-- NotADirectoryError
|   +-- PermissionError
|   +-- ProcessLookupError
|   +-- TimeoutError
+-- ReferenceError
+-- RuntimeError
|   +-- NotImplementedError
|   +-- RecursionError
+-- SyntaxError
|   +-- IndentationError
|   +-- TabError
+-- SystemError
+-- TypeError
+-- ValueError
|   +-- UnicodeError
|       +-- UnicodeDecodeError
|       +-- UnicodeEncodeError
|       +-- UnicodeTranslateError

```

Python内置的异常



```

+-- Warning
+-- DeprecationWarning
+-- PendingDeprecationWarning
+-- RuntimeWarning
+-- SyntaxWarning
+-- UserWarning
+-- FutureWarning
+-- ImportWarning
+-- UnicodeWarning
+-- BytesWarning
+-- EncodingWarning
+-- ResourceWarning

```



处理异常

- **合理地**使用异常处理结果可以使得程序更加健壮，具有更强的容错性
 - 不会因为用户不小心的错误输入或其他运行时原因而造成程序终止
 - 可以使用异常处理结构为用户提供更加友好的提示
- *有些代码执行可能会出现错误，也可能不会出现错误，这主要由运行时的各种客观因素决定，此时建议使用异常处理结构。如果使用大量的选择结构来提前判断，仅当满足相应条件时才执行该代码，这些条件可能会严重干扰正常的业务逻辑，也会降低代码的可读性。*



处理异常

- 出现异常的两种场景：
 - 第一个场景：错误发生但未处理；
 - 当一个错误发生了，异常被打印出来，称为未处理异常，这时因为没有代码来明确地关注和处理异常，异常被缺省处理，自动输出一些调试信息并终止运行。
 - 第二个场景：具备检测与处理
 - 通过代码去明确地处理异常
 - 程序不会终止运行，并能增强容错性。



异常处理

- 尽管异常处理机制非常重要也非常有效，但是不建议使用异常来代替常规的检查，例如必要的if...else判断等。
- 在编程时应避免过多依赖于异常处理机制来提高程序的健壮性。
- 程序出现异常或错误之后是否能够调试程序并快速定位和解决存在的问题也是程序员综合水平和能力的重要体现方式之一



错误如何处理

先尝试

处理[多组]异常继续进行

没有异常则进入后继处理

无论有无异常均进入最终处理

try

except

else

finally



Python中的异常处理结构

try:

<tryBody> #被监控的代码
[except [expression [as identifier]]:] # 至少一次, 在多次出现时不带表达式except应该是最后一个。按序匹配找到对应的exception为止

<exceptionBody> #异常处理代码
except # 可选的

except # 可选的

else: # 可选的, 没有异常出现时执行
<elseBody>

finally: # 可选的, 不管异常有没有都要执行
<finallyBody>

- **expression**应该为
 - 异常类
 - 或者**异常类的元组**,
表示其中任一异常出现

部分代码块为可选



1

```
try  
finally
```

2

```
try  
except
```

3

```
try  
except  
finally
```

4

```
try  
except  
else  
finally
```

错误如何处理



尝试处理

无论有无异常均进入最终处理

try

finally



Python中的异常处理结构

- 执行<tryBody>代码
- 不管有没有异常都会执行<finallyBody>块

```
try:  
    <tryBody> #被监控的代码  
finally:  
    <finallyBody>
```

```
>>> try:  
        print(A)  
finally:  
    print("finally run!")
```

```
finally run!  
Traceback (most recent call last):  
  File "<pyshell#17>", line 2, in <module>  
    print(A)  
NameError: name 'A' is not defined
```

错误如何处理



尝试处理

处理异常继续进行

try

except



try.....except结构

- 这是最基本的处理结构；
- 具有两种形式。

代码处理形式一

```
try:  
try_block #被监控的代码  
#except BaseException:  
except Exception: #使用异常基类，捕获所有的异常  
except_block #异常处理代码  
优势：能够处理所有的异常
```

代码处理形式二

```
try:  
try_block #被监控的代码  
except Exception1: #使用特定异常类  
except_block #针对性异常处理代码
```

- **建议**

- 尽量显式捕捉可能会出现的异常，并编写具有针对性的代码；
- 但可能无法覆盖所有的异常

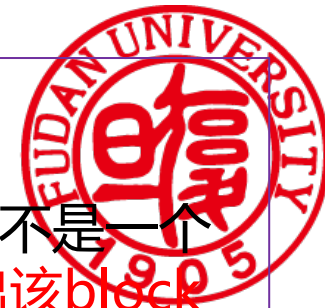


try.....except : 示例1

```
while True:
    try:
        x = int(input("Please enter a number: "))
        break
    except ValueError:
        print("That was no valid number. Try again...")
```

Please enter a number: a
That was no valid number. Try again...
Please enter a number: b
That was no valid number. Try again...
Please enter a number: c
That was no valid number. Try again...
Please enter a number: 10

try.....except : 示例 2 (不作要求)



expression as **instance**

instance为异常对象

补充内容：在exception block并不是一个新的scope，但是**instance**在退出该block后不可用

```
try:
    raise Exception('spam', 'eggs')
except Exception as inst:
    print(type(inst))    # the exception instance
    print(inst.args)     # arguments stored in .args
    print(inst)          # __str__ allows args to be printed directly,
                        # but may be overridden in exception subclasses
    x, y = inst.args     # unpack args
    print('x =', x)
    print('y =', y)
    print(x)
    print(inst)
```

```
<class 'Exception'>
('spam', 'eggs')
('spam', 'eggs')
x = spam
y = eggs
spam
Traceback (most recent call last):
  File "/Users/bxliu/Code/pythonProject/1.py", line
  12, in <module>
    print(inst)
NameError: name 'inst' is not defined
```

错误如何处理



先尝试

try

处理异常继续进行

except

没有异常则进入后

else

继处理



try.....exceptelse : 示例1

```
a_list = ['China', 'America', 'England', 'France']
while True:
    print('请输入字符串的序号')

    try:
        n = int(input())
        print(a_list[n])
    except IndexError:
        print('列表元素的下标越界，请重新输入字符串的序号')
    else:
        break
```

Copy&Run

- 分析上述的代码
 - 输入错误：例如输入的数值超出了正常的序号，会产生异常；执行红色print语句，并继续循环；
 - 没有异常时：输出列表中正确的字符串，并且退出循环。
 - 但是输入了非数值？ → 没有捕获异常，程序退出



带有多个except的try结构

```
try:  
    try_block      #被监控的语句  
except Exception1:  
    except_block_1  #处理异常1的语句  
except Exception2:  
    except_block_2  #处理异常2的语句  
...
```

- 功能：

- 根据实际产生的异常，执行其中的异常程序块。
- 按照顺序匹配，一旦匹配执行异常程序块，后面的except块不再执行

带有多个except的try结构 示例1



```
a_list = ['China', 'America', 'England', 'France']
while True:
    print('请输入字符串的序号')
    try:
        n = int(input())
        print(a_list[n])
    except IndexError:
        print('列表元素的下标越界，请重新输入字符串的序号')
    except ValueError:
        print('输入不是十进制整数，请重新输入字符串的序号')
    else:
        break
```



带有多个except的try：示例2

- 典型例子：除法。

try:

```
x=eval(input('请输入被除数: '))
```

```
y=eval(input('请输入除数: '))
```

```
z= x / y
```

```
except ZeroDivisionError:
```

```
    print('除数不能为零')
```

```
except TypeError:
```

```
    print('被除数和除数应为数值类型')
```

```
except NameError:
```

```
    print('变量不存在')
```

```
except Exception as e:
```

```
    print('其他错误:',e)
```

```
else:
```

```
    print(x, '/', y, '=', z)
```

- 建议2

- 最后一个except用来捕捉Exception (BaseException) 。

带有多个except的try : 示例3



功能：打开文件，读一行内容，将其转换为整数，并且输出

```
def parse_int(file='sample.txt'):
    try:
        f = open(file)
        s = f.readline() #如果这一行执行时出错怎么办
        f.close()
        i = int(s.strip())
        print(i)
    except (OSError, ValueError) as inst:
        print(type(inst),inst)
    except:
        print("Unexpected error:", sys.exc_info()[0])
```

```
if __name__ == '__main__':
    setup() # 创建测试环境，生成相应的文件
    try:
        parse_int('sample.txt') #该文件的第一行为整数，正常运行输出结果
        parse_int('sample1.txt') # 该文件不存在,匹配OSError
        parse_int('sample2.txt') #该文件第一行为非整数，触发异常ValueError
        parse_int('sample3.txt') # 该文件以utf-8编码, readline()出错，
            # 匹配ValueError，但文件并没有关闭
    except Exception as e:
        print(e)
```

不管是正常执行，还是异常出现时都希望能够关闭打开的文件

错误如何处理



先尝试

try

处理异常继续进行

except

无论有无异常均进

finally

入最终处理



try.....exceptfinally结构

- 特点

- Finally中的语句总会执行；
- 可以用于清理工作，以便释放资源。

- 典型结构如下

try:
 try_block #被监控的代码
except:
 except_block #例外处理程序块
finally:
 finally_block #无论如何都会执行

```
>>> try:  
    print(A)  
  
except:  
    print("capture the error:")  
finally:  
    print("finally run!")
```

```
capture the error:  
finally run!
```

```
>>> try:  
    print(A)  
  
except Exception as e:  
    print("capture the error:" ,e)  
finally:  
    print("finally run!")
```

```
capture the error: name 'A' is not defined  
finally run!
```

```
>>> try:  
    print(A)  
  
except NameError as e:  
    print("capture the error:" ,e)  
finally:  
    print("finally run!")
```

```
capture the error: name 'A' is not defined  
finally run!
```



try.....exceptfinally : 示例1 (1)

- 典型例程：文件的读取
- 仅仅能防止文件读取操作出现异常而导致文件不能正常关闭

```
try:
    f = open('sample2.txt', 'r')
    line = f.readline()
    print(line)
finally:
    print("finally running")
    f.close()
```

```
>>> try:
      f = open('sample2.txt', 'r')
      line = f.readline()
      print(line)
    finally:
      print("finally running")
      f.close()
```

```
finally running
Traceback (most recent call last):
  File "<pyshell#1>", line 2, in <module>
    f = open('sample2.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory: 'sample2.txt'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "<pyshell#1>", line 7, in <module>
    f.close()
NameError: name 'f' is not defined
>>>
```

- 如果文件没有创建，则在finally中会产生异常。

8.3.4 try.....exceptfinally结构: 示例1

```
def parse_int2(file='sample.txt'):
    try:
        f = open(file)
        s = f.readline()
        # f.close()
        i = int(s.strip())
        print(i)
    except (OSError, ValueError) as inst:
        print(type(inst), inst)
    except:
        print("Unexpected error:", sys.exc_info()[0])
        raise
    finally:
        f.close()
```

如果open时异常
? f未定义, 从而抛出NameError

```
try:
    with open(file) as f:
        s = f.readline()
        i = int(s.strip())
        print(i)
except (OSError, ValueError) as inst:
    print(type(inst), inst)
except:
    print("Unexpected error:", sys.exc_info()[0])
    raise
```

```
def parse_int3(file='sample.txt'):
    f = None
    try:
        f = open(file)
        s = f.readline()
        # f.close()
        i = int(s.strip())
        print(i)
    except (OSError, ValueError) as inst:
        print(type(inst), inst)
    except:
        print("Unexpected error:", sys.exc_info()[0])
        raise
    finally:
        if f:
            f.close()
```

建议采用with block打开文件, 不用管文件是否关闭的问题



try.....exceptfinally : 示例2

- 如果try子句中的异常没有被捕获与处理，或者except子句、else子句中的代码出现了异常，那么这些异常将会在finally中的语句执行完后再次抛出。

```
def divide(x, y):  
    try:  
        result = x / y  
    except ZeroDivisionError:  
        print("division by zero!")  
    else:  
        print("result is", result)  
    finally:  
        print("executing finally clause")
```

```
>>> divide(2, 1)  
result is 2.0  
executing finally clause  
>>> divide(2, 0)  
division by zero!  
executing finally clause  
>>> divide("2", "1")  
executing finally clause  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
  File "<stdin>", line 3, in divide  
TypeError: unsupported operand type(s) for /:  
'str' and 'str'
```



try.....exceptfinally : 示例3

- **finally代码中：应该尽量避免使用return语句**
- 在finally字句中有return或者break时会取消要抛出的异常！

```
def demo_div(a, b):  
    try:  
        return a/b  
    except ZeroDivisionError:  
        pass  
    finally:  
        return -1
```

```
>>> demo_div(1, 0)  
-1  
>>> demo_div("1","2") #虽然有异常，但是被取消了  
-1  
>>> demo_div(1, 2)  
-1  
>>> demo_div(10, 2)  
-1
```



异常处理总结

- 可以通过raise子句来抛出异常
- except子句用于捕获指定的异常，按照顺序匹配，一旦匹配，后面的except子句跳过
- except子句可以通过 as instance来获得异常对象，但是注意except子句退出后instance不再有效
- 可选的else子句在没有异常时执行
- 可选的finally子句不管有没有异常都会执行
- 如果没有捕获异常，则该异常会往try外层抛出
 - 在finally字句中有return或者break时会取消要抛出的异常！



断言与上下文处理

- 断言与上下文处理
 - 两种特殊的异常处理形式；
 - 形式上比通常的异常处理简单；

错误如何处理



对特定条件

真：继续进行

假：抛出异常

assert expression

断言(不作要求)

```
if __debug__:
    if not expression:
        raise AssertionError
    # raise AssertionError(reason)
```

- 断言语句的语法

`assert expression[,reason]`

- 当判断表达式expression为真时，什么都不做；如果表达式为假，则抛出异常。
- 即要求expression为真,用于开发程序时对特定必须满足的条件进行验证
- 什么时候__debug__为True?
 - 目前的实现中，正常情况为True
 - 当Python脚本以-O选项编译为字节码文件时，__debug__为False，assert语句不会产生任何代码



断言：示例1

```
try:  
    assert 1 == 2, "1 is not equal 2!"  
except AssertionError as reason:  
    print("%s:%s"%(reason.__class__.__name__, reason))
```

AssertionError:1 is not equal 2!

断言：示例2



```
def assert_divzero( ):
    a = int(input('请输入被除数:'))
    b = int(input('请输入除数:'))
    assert b!=0, '除数不能为0!'
    c = a/b
    print(a,'/',b,'=',c)
```

```
>>> assert_divzero()
请输入被除数:15
请输入除数:0
Traceback (most recent call last):
  File "<pyshell#142>", line 1, in <module>
    assert_divzero()
  File
"C:/Users/dlmao/PycharmProjects/pythonC
lass/exception_demo.py", line 190, in
assert_divzero
    assert b!=0, '除数不能为0!'
AssertionError: 除数不能为0!
```

错误如何处理



进入一个特定结构 (context)

(自动进行资源管理)

with



上下文管理

- 使用with语句进行上下文管理，在with_block结束时释放资源
- 避免了try...finally结构中的资源释放问题
- 不管异常是否出现都会释放在进入时获得的资源

```
with context_expr [as obj]:  
    with_block
```

用户也可以自己实现某个资源的上下文管理（**不作要求**）

依赖于python语言的magic method，需要实现__enter__()和__exit__()两个方法：**上下文管理协议**；

或者通过引用contextlib，并使用@contextlib.contextmanager方式实现。

with语句等价于如下代码

```
obj = context_expr  
obj.__enter__()  
try:  
    with_block  
finally:  
    obj.__exit__()
```

with语句使用：示例



1. 显示文件内容

```
def read_file_with(file='sample.txt'):
    with open(file) as f:
        for line in f:
            print(line,end="")
```

2. 拷贝文件file到file-bk

```
def copy_file_with2(file='sample.txt'):
    with open(file) as fr:
        with open(file+'-bk',"w") as fw:
            for line in fr:
                fw.write(line)
```

```
def copy_file_with(file='sample.txt'):
    with open(file) as fr, open(file+'-bk',"w") as fw:
        for line in fr:
            fw.write(line)
```

错误如何处理



主动抛出异常

raise Exception



抛出异常 (raise)

- 当程序出现错误，python会自动引发异常，也可以通过raise显式地引发异常。
- 主动抛出异常 `raise [expression [from expression2]]`
 - 定义自己的异常类时；
 - 或者需要抛出异常时。

```
if type(score) is str:  
    raise ValueError("score 类型有错")
```



抛出异常raise语句

- raise语法

- expression : 可以是一个异常类的实例对象 ; 也可以是异常类 , 系统会首先创建一个该类的实例 , 然后抛出该异常对象 ; :
- from expression2:给出了将两个异常连接起来的方法 , 表示异常是由于另一个expression2引起的 (不作要求) 。

```
raise Exception
raise Exception('foo occurred!' )
if type(score) is str:
    raise ValueError("score 类型有错")
raise Exception('foo occurred!') from Exception('another error
occured!')
```



raise例子

```
>>> int("12eef")|
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    int("12eef")
ValueError: invalid literal for int() with base 10: '12eef'
```

```
>>> score="123"
>>> if type(score) is str:
    raise ValueError("score 类型有错")
```

```
Traceback (most recent call last):
  File "<pyshell#7>", line 2, in <module>
    raise ValueError("score 类型有错")
ValueError: score 类型有错
```

错误如何处理



自定义Exception
以提供更灵活处理



继承实现自定义异常（不作要求）

- 下面的例子演示了自定义的异常类
 - 必须继承Exception类或者其子类：Exception为所有异常类的基类；

```
class MyError(Exception):  
    def __init__(self, value):  
        self.value = value  
    def __str__(self):  
        return repr("My error:%d"%self.value)
```

```
try:  
    raise MyError(2*2)  
except MyError as e:  
    print('My exception occurred', e)  
    raise
```

```
My exception occurred 'My error:4'  
Traceback (most recent call last):  
  File "<pyshell#4>", line 2, in <module>  
    raise MyError(2*2)  
MyError: 'My error:4'  
>>>
```



课程后继安排



课程后继安排

1. 重点回顾与梳理
2. 模拟卷
3. 答疑

协助大家完成期中作业

原定团队协作作业简化进行



课堂练习与作业答疑

无需提交

TODOLIST



- 结合上次课堂作业内容和现有基础上，给TODOLIST增加文件处理和操作日志功能：
- 程序启动和停止，均**记录一行当前时间**（追加）至start.log；
- 程序启动时，提问用户名，用户输入用户名后，检索list.conf中记录的用户名和记录文件名，
 - 如果未检索到，创建一个记录文件，并增加至list.conf中记录，
 - 如果检索到，加载文件中的TODO信息，并打印出来，此后用户可以继续进行记录操作；
- 程序结束时，将用户所有的TODOLIST保存至用户自己的记录文件中
- 给OP功能选择增加保存功能，
 - 输入**SAVE**，则等待用户输入一个文件名，并将用户TODOLIST保存在其中，如果文件操作出现异常，请用户重新输入；
- 将打印待完成事项的第几项，**修改为异常处理模式（try except finally）**，**可以处理输入异常和索引越界异常**
- **思考如何让自己的程序健壮性更强**